

认知科学与类脑计算

主讲教师：刘千惠

联系方式：qhliu@sdu.edu.cn

山东大学 人工智能学院



电压驱动的单层Tempotron学习算法

- 扩展Tempotron至多分类场景：

假设有 C 个类别，就设置 C 个输出神经元，每个输出神经元对应一个类别。

类别	输出神经元
猫	neuron 1
狗	neuron 2
鸟	neuron 3

训练时，样本属于第 k 类，则：

第 k 个神经元应该发放

其他神经元不应该发放

$$y_k = 1$$

$$y_j = 0, \quad j \neq k$$

电压驱动的单层Tempotron学习算法

- 扩展Tempotron至多分类场景:

对第j个输出神经元, 计算膜电位:

$$V_j(t) = \sum_i w_{ji} \sum_{t_i^f < t} K(t - t_i^f)$$

判断它是否发放:

$$\max_t V_j(t) \geq \theta$$

对于正确类别神经元 $j=k$

如果它没有发放:

$$\Delta w_{ki} = +\eta \sum_{t_i^f < t_{k,\max}} K(t_{k,\max} - t_i^f)$$

对于错误类别神经元 $j \neq k$

如果它发放了:

$$\Delta w_{ji} = -\eta \sum_{t_i^f < t_{j,\max}} K(t_{j,\max} - t_i^f)$$

电压驱动的单层Tempotron学习算法

- 扩展Tempotron至多分类场景:

推理时可按照神经元发放情况或膜电位大小决定类别

1. 哪个神经元发放（早）就选哪个

理想情况：第 k 个神经元发放，其他不发放，则输出类别为 k 。

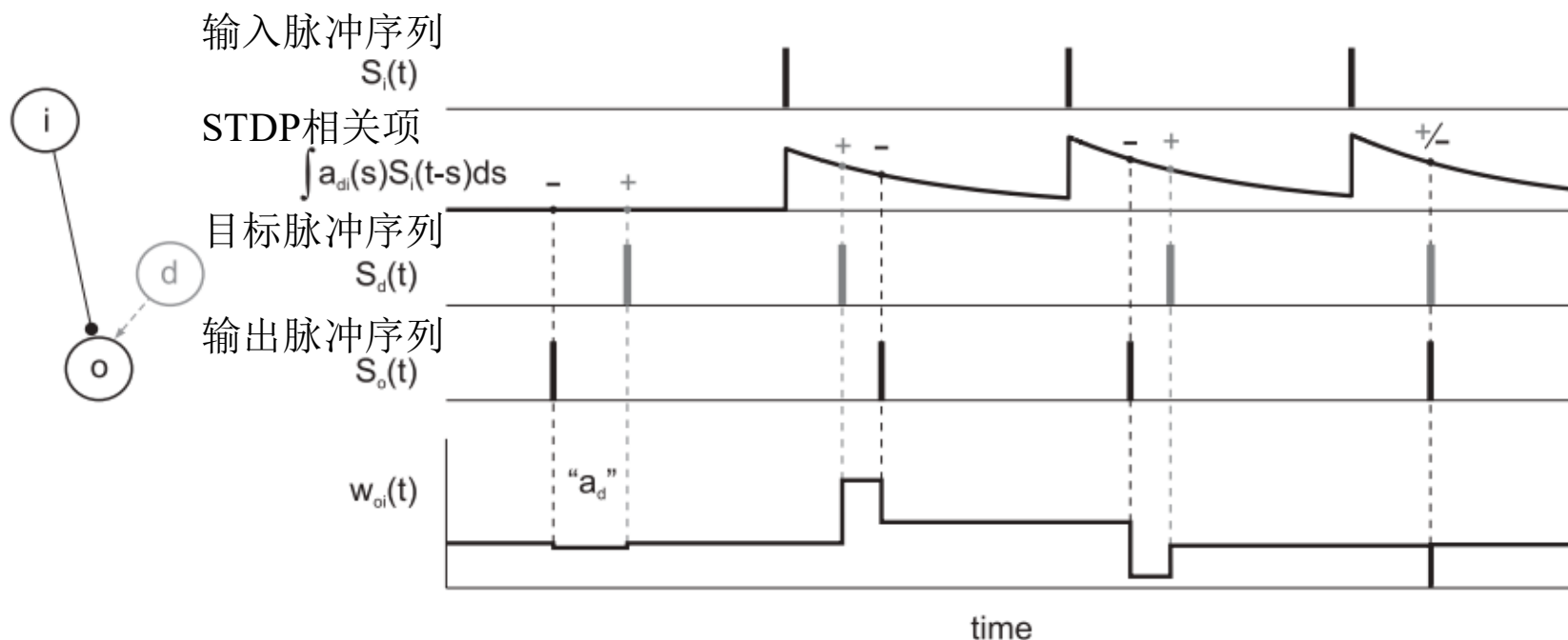
如有多个神经元发放，则选择发放最早的神经元对应的类别。

2. 选最大膜电位的类别

即使没有神经元真正越过阈值，也可以选峰值最高的那个类。

远程监督学习算法(Remote Supervised Method, ReSuMe)

- 突触权值的调整仅依赖于输入输出的脉冲序列和STDP机制，与神经元模型和突触类型无关，因此该算法可用于各种神经元模型。



远程监督学习算法(Remote Supervised Method, ReSuMe)

$$\frac{d}{dt}w_{oi}(t) = [S_d(t) - S_o(t)] \left[a_d + \int_0^\infty a_{di}(s) S_i(t-s) ds \right].$$

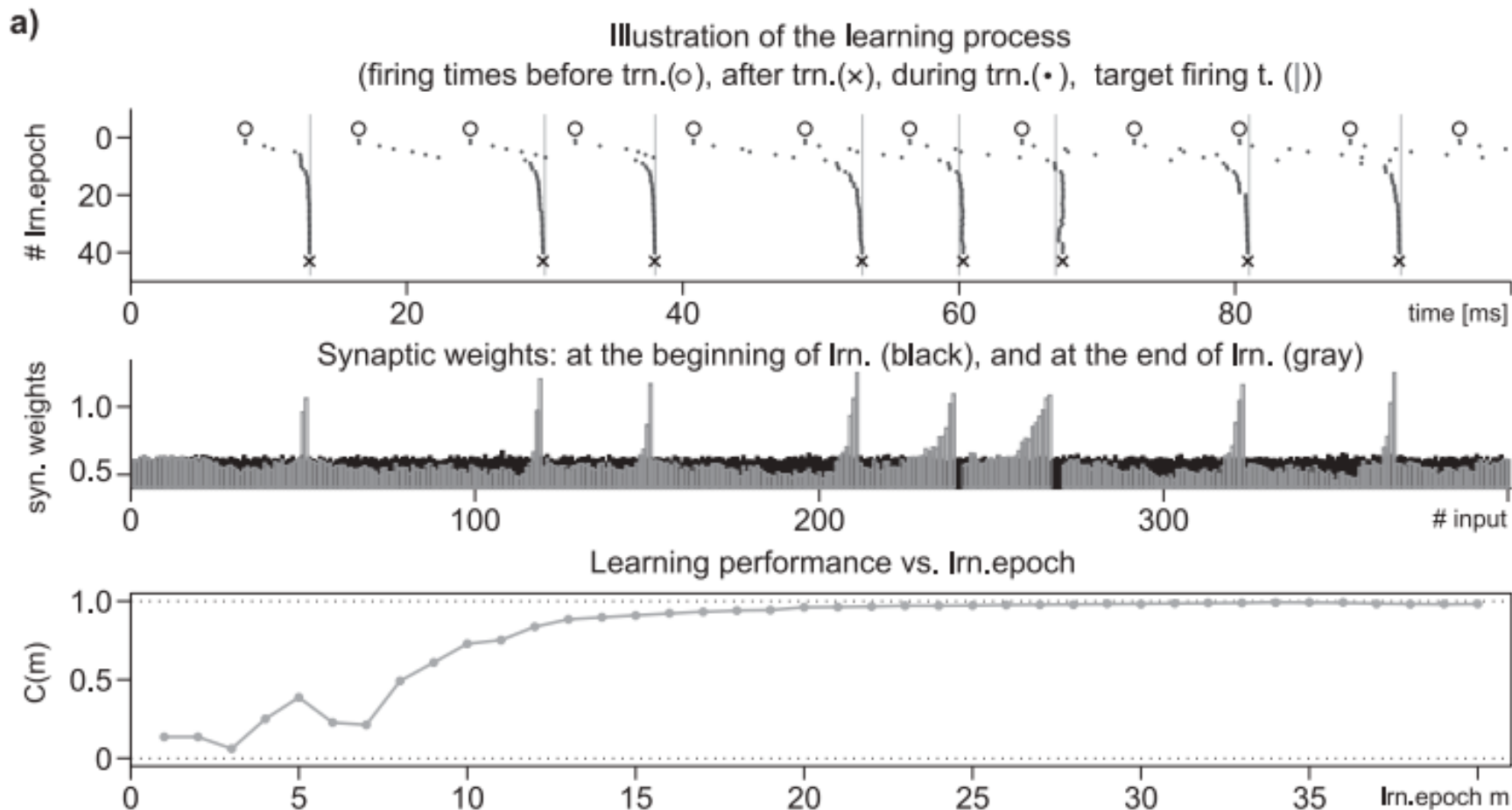
$S(t) = \sum_f \delta(t - t^f)$, where $f = 1, 2, \dots$ 为脉冲标号。

α_d : 主要用于调节输出神经元的平均发放率, 使实际输出的平均 firing rate 接近目标。

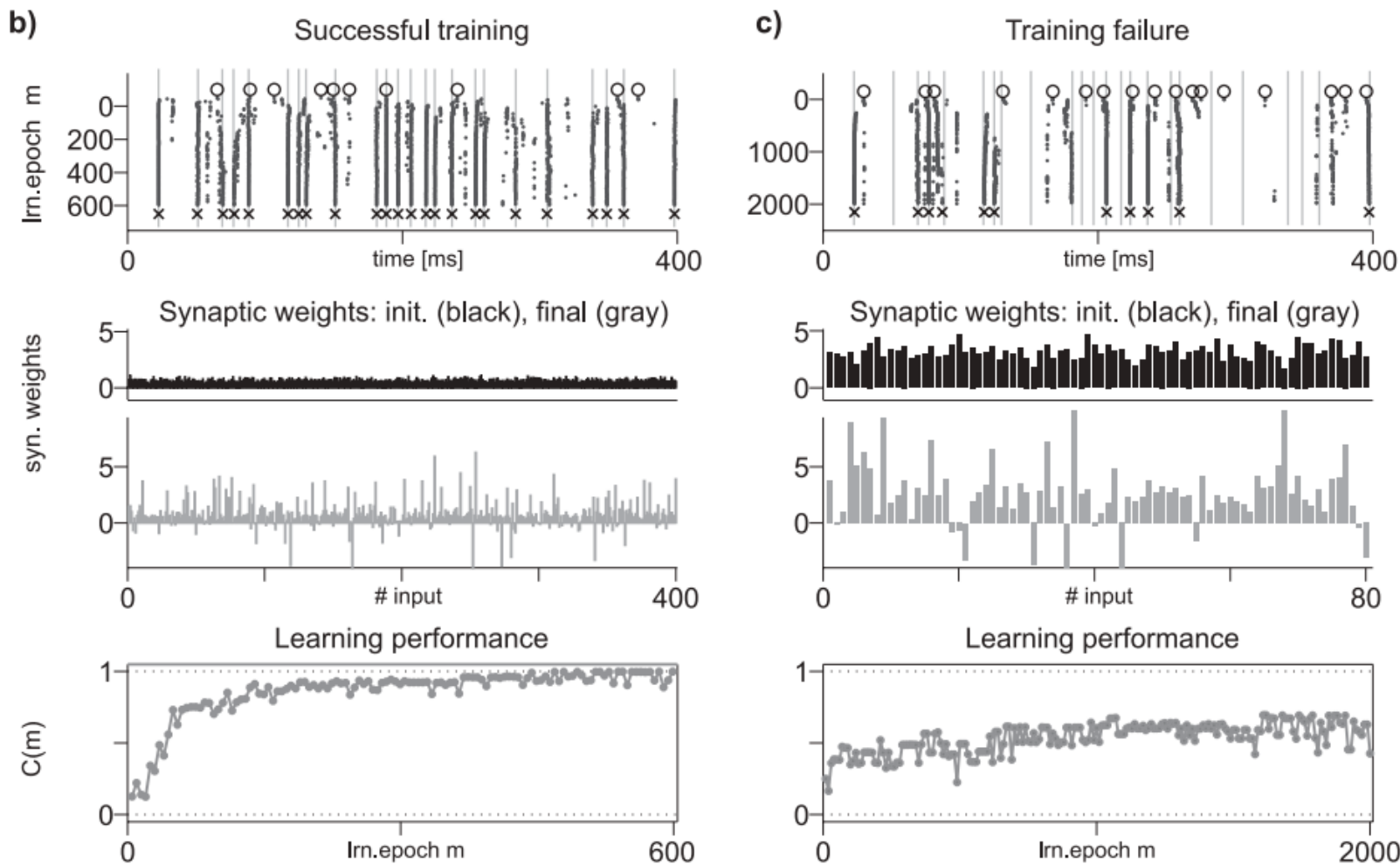
$\alpha_{di}(s)$: 学习窗口, 它定义在突触位点脉冲发生的时间延迟 s 内。

$$a_{di}(s) = A_{di} \exp\left(-\frac{s}{\tau_{di}}\right), \quad s \geq 0$$

远程监督学习算法(Remote Supervised Method, ReSuMe)



远程监督学习算法(Remote Supervised Method, ReSuMe)



脉冲神经网络监督学习算法

- 经典人工神经网络的学习算法回顾
- 单层脉冲神经网络的学习算法
- 深度脉冲神经网络的学习算法
- 转换算法（ANN-SNN Conversion）

浅层脉冲神经网络的结构相对简单，参数较少，信息的传递和转换层次较少，因而在一些简单的任务上，比如模式识别、简单时间序列分类等，表现出色。然而，随着任务复杂度的增加，浅层SNN在学习能力和表达能力上逐渐显现出局限性，难以捕捉更复杂的时间与空间特征。相比之下，深度脉冲神经网络（Deep Spiking Neural Networks, DSNNs）具有更丰富的参数量和更深的网络结构，能够捕获更加抽象和复杂的特征表示，从而在目标检测、运动追踪、复杂时间序列预测等任务中展现出强大的建模能力。

深度脉冲神经网络学习算法



山东大学
SHANDONG UNIVERSITY

	浅层SNN	深层SNN
隐藏层数量	1~2	≥ 3
参数量	较少	较多
表征能力	只能表征简单特征	可以表征复杂特征
训练难度	小数据集，容易收敛	大数据集，较难收敛
鲁棒性	弱	强
能耗	低	高

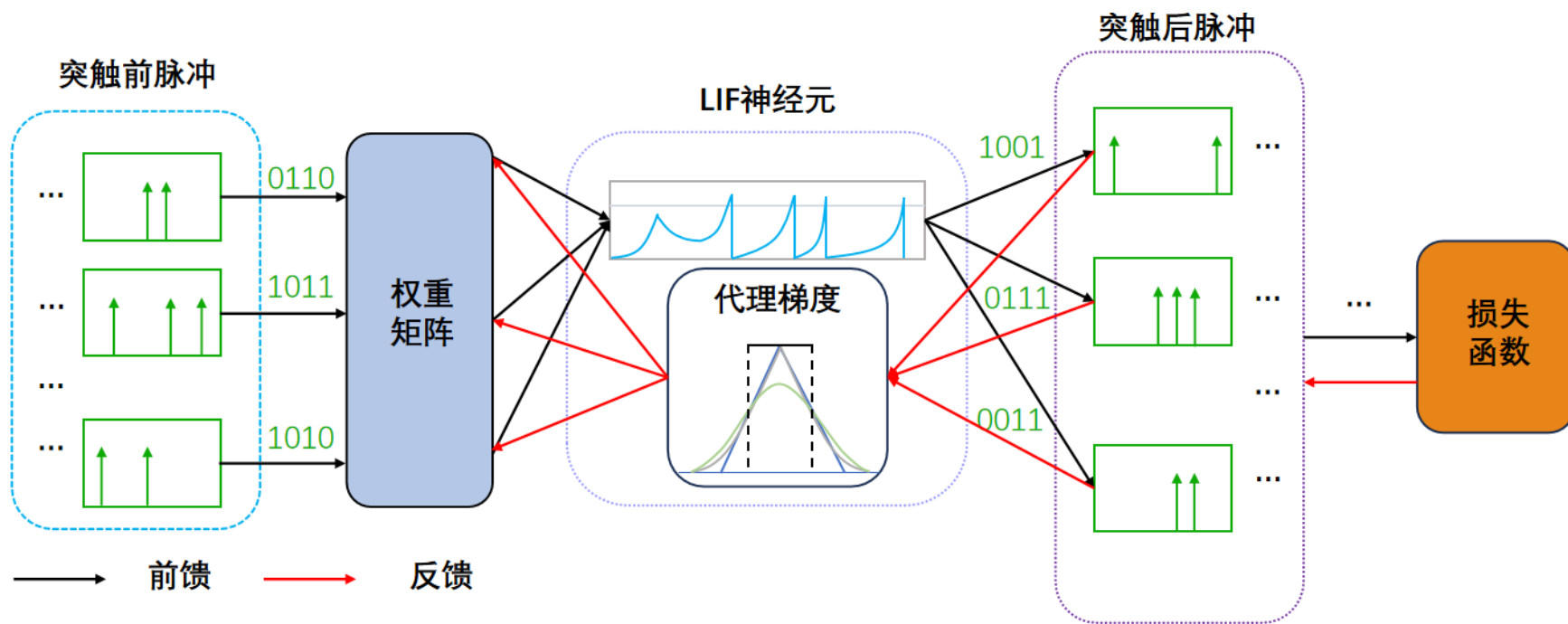
深度SNN在训练过程中面临诸多挑战：

- **梯度消失与梯度爆炸：** 随着网络层数的增加，梯度在反向传播过程中可能会逐层减小或增大，导致网络训练难以收敛。
- **时间维度上的依赖性：** 脉冲神经网络需要在时间域内进行计算，导致梯度计算和权重更新更加复杂。
- **高计算成本：** 深度SNN在时间和空间上需要更高的计算资源，尤其是在大规模数据集上的训练过程中。
- **数据稀疏性：** 脉冲神经网络以离散脉冲事件为单位进行信息传递，数据的稀疏性给训练带来了额外的难度。

误差反向传播是充分利用多时间尺度信息训练深层脉冲神经网络最有前景的方法之一。然而，脉冲发放过程的不可微特性，阻碍了反向传播在脉冲神经网络（SNN）中的直接应用。为了应对这一挑战，研究领域发展出两类基于梯度的训练方法：

- 基于代理梯度学习方法
- 基于脉冲发放时间学习方法

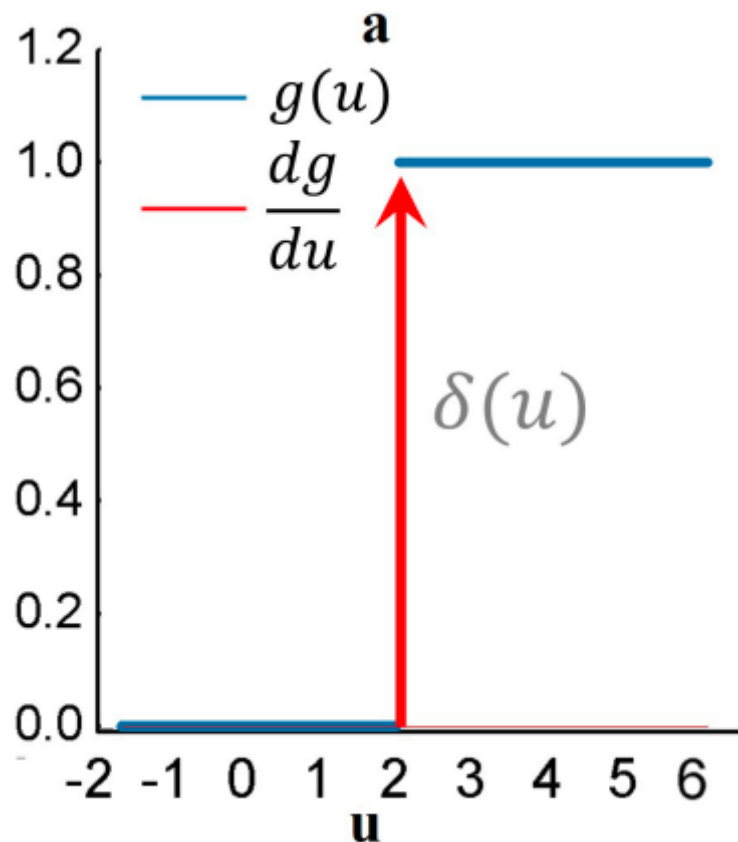
基于代理梯度的反传



基于代理梯度的反传

$$g(x) = \begin{cases} 1, & x \geq V_{th} \\ 0, & x < V_{th} \end{cases}$$

它的导数在0处取无限值，其余地方都取0，会在误差传播过程中造成梯度消失或爆炸的问题，导致梯度无法在 SNN 的网络层间有效传递，阻碍了梯度流动与反向传播过程。

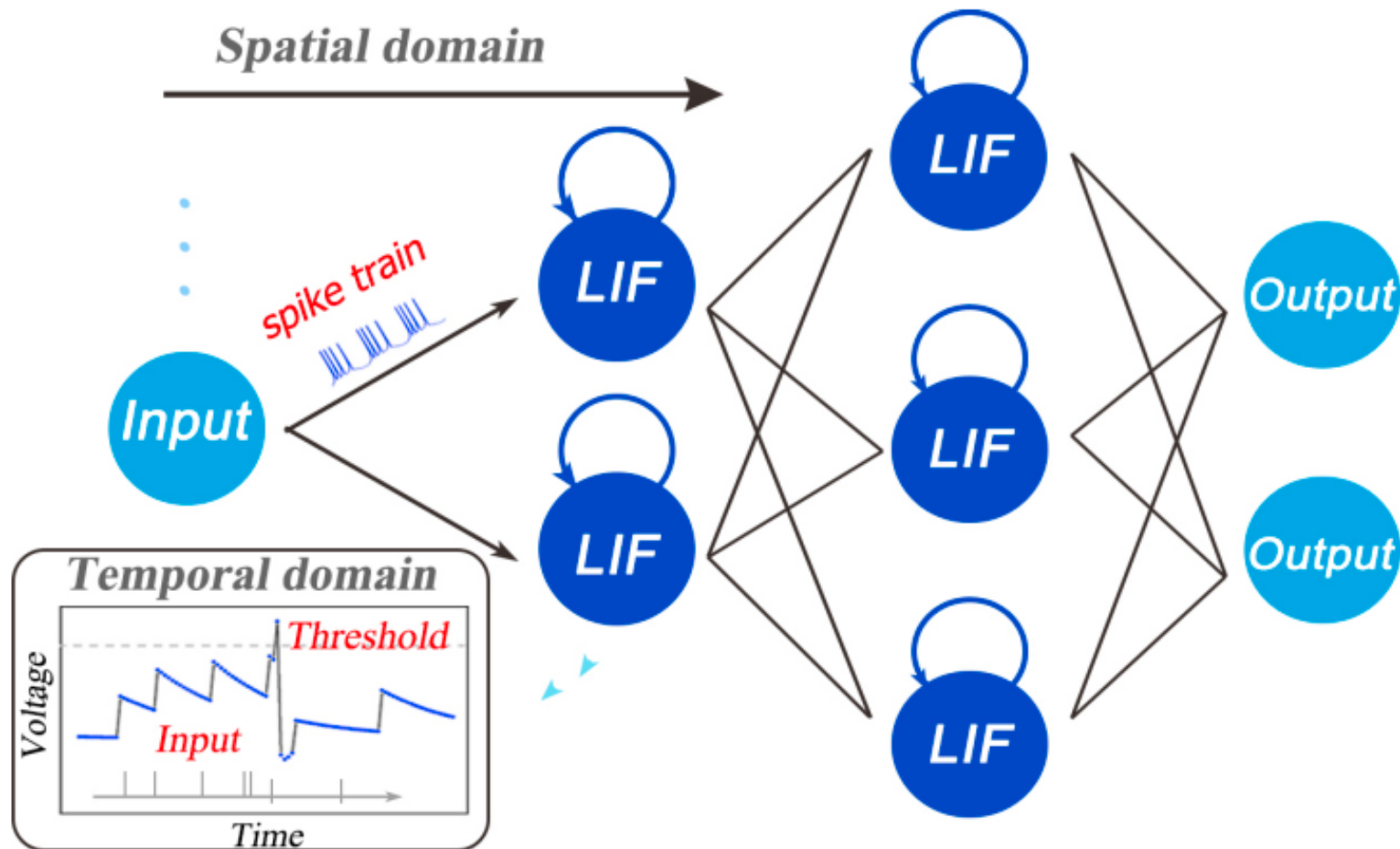


基于代理梯度的反传

代理梯度方法采用平滑曲线，来估计脉冲神经网络中不可微阶跃函数的导数。通过这种“伪梯度”方式，反向传播得以在脉冲神经网络的空间和时域中迭代实现。

替代梯度学习可以充分利用深度学习的完整生态，因此被广泛应用于视觉识别、文本处理、语音分析等具有挑战性的任务中。

Spatio-Temporal Backpropagation (STBP) 算法



Spatio-Temporal Backpropagation (STBP) 算法

STBP算法同时考虑了SNN在时间和空间维度的误差反传，并将代理梯度函数引入到深度SNN训练中，建立了适合梯度下降训练的深度SNN时空反传算法。

1. 迭代LIF模型
2. 时空反向传播训练框架
3. 代理梯度函数

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

LIF模型的微分方程形式如下：

$$\tau \frac{du}{dt} = -u + I$$

其中， u 表示神经元在 t 时刻的膜电压， τ 代表时间常数， I 表示突触前输入，由前一层神经元的输出和突触权重决定。当膜电压 u 超过给定阈值 V_{th} 时，神经元会发出一个脉冲，并将其电位重置为 u_{reset} 。

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

对LIF模型适当简化，并且遵循事件驱动迭代的更新规则，
有下式：

$$u(t) = u(t_{i-1})e^{\frac{t_{i-1}-t}{\tau}} + I(t)$$

神经元电位 $u(t)$ 可以利用末次放电时刻 t_{i-1} 和突触前输入 $I(t)$ 来近似。膜电位呈指数衰减，直到神经元接收到突触前输入，一旦神经元发出脉冲，新的更新周期就会开始。也就是说，神经元状态是由 $I(t)$ 的空间累积和 $u(t_{i-1})$ 的漏电共同决定的。

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

对LIF模型适当简化，并且遵循事件驱动迭代的更新规则，
有下式：

$$u(t) = u(t_{i-1})e^{\frac{t_{i-1}-t}{\tau}} + I(t)$$

神经元电位 $u(t)$ 可以利用末次放电时刻 t_{i-1} 和突触前输入 $I(t)$ 来近似。膜电位呈指数衰减，直到神经元接收到突触前输入，一旦神经元发出脉冲，新的更新周期就会开始。也就是说，神经元状态是由 $I(t)$ 的空间累积和 $u(t_{i-1})$ 的漏电共同决定的。

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

使用误差反向传播训练 DNN 的高效性，很大程度上得益于梯度下降的迭代表示形式；这种形式在空间域反向传播过程中产生了链式法则，从而实现逐层误差传播。

STBP提出一种基于迭代 LIF 神经元的 SNN，其中迭代同时发生在空间域和时间域。

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

$$x_i^{t+1,n} = \sum_{j=1}^{l(n-1)} w_{ij}^n o_j^{t+1,n-1}$$

$$u_i^{t+1,n} = u_i^{t,n} f(o_i^{t,n}) + x_i^{t+1,n} + b_i^n$$

$$o_i^{t+1,n} = g(u_i^{t+1,n})$$

W_{ij} 为从突触前第 j 个神经元到突触后第 i 个神经元的突触权重, $o_j \in \{0,1\}$ 表示第 j 个神经元的输出。 x_i 是第 i 个神经元突触前输入的简化表示, 类似于原始 LIF 模型中的 I 。 u_i 是第 i 个神经元的膜电位, b_i 是偏置参数。

Spatio-Temporal Backpropagation (STBP) 算法

1. 迭代LIF模型

使用遗忘门 $f(\cdot)$ 控制 TD 记忆，并使用输出门 $g(\cdot)$ 触发脉冲。

遗忘门 $f(\cdot)$ 控制 TD 中潜在记忆的泄漏程度，输出门 $g(\cdot)$ 在激活时产生脉冲活动。具体而言，对于较小的正时间常数 T ， $f(\cdot)$ 可以近似为

$$g(x) = \begin{cases} 1, & x \geq V_{th} \\ 0, & x < V_{th} \end{cases} \quad f(o_i^{t,n}) \approx \begin{cases} \tau, & o_i^{t,n} = 0 \\ 0, & o_i^{t,n} = 1 \end{cases}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

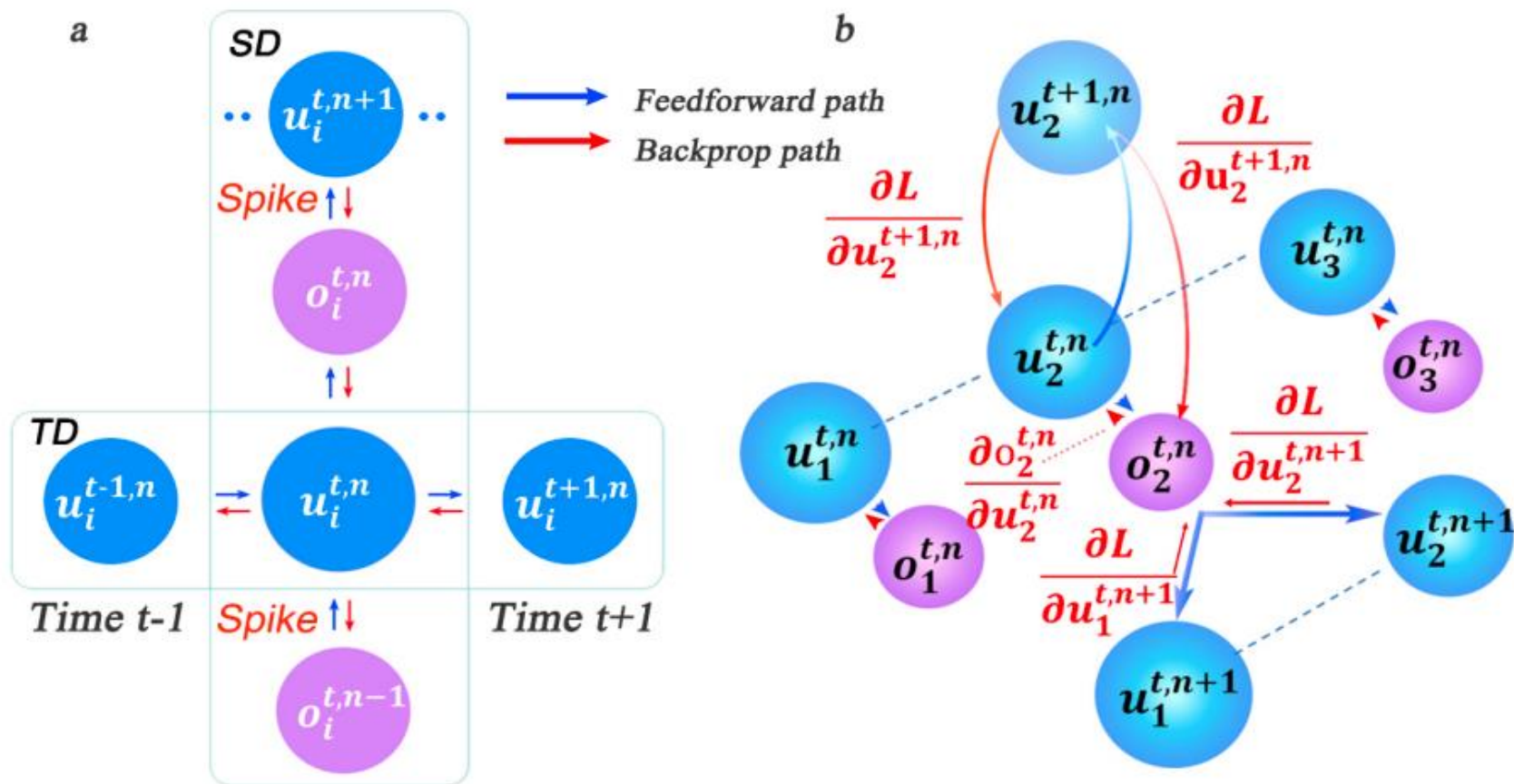
为了提出STBP训练框架，该方法首先定义了损失函数 L ，最小化给定时间窗口 T 下所有样本的均方误差。

$$L = \frac{1}{2S} \sum_{s=1}^S \left\| y_s - \frac{1}{T} \sum_{t=1}^T \mathbf{o}_s^{t,N} \right\|_2^2$$

y_s 和 $\mathbf{o}_s$ 分别表示第 s 个训练样本的标签向量和最后一层 N 的输出向量。通过迭代LIF模型的微分方程可以看出 L 是 W 和 b 的函数。因此梯度下降需要获得 L 对 W 和 b 的导数。

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架



Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 1: $t = T$ at the output layer $n = N$.

In this case, the derivative $\frac{\partial L}{\partial o_i^{T,N}}$ can be directly obtained since it depends on the loss function in Eq.(9) of the output layer. We could have

$$\frac{\partial L}{\partial o_i^{T,N}} = -\frac{1}{TS}(y_i - \frac{1}{T} \sum_{k=1}^T o_i^{k,N}). \quad (11)$$

The derivation with respect to $u_i^{T,N}$ is generated based on $o_i^{T,N}$

$$\frac{\partial L}{\partial u_i^{T,N}} = \frac{\partial L}{\partial o_i^{T,N}} \frac{\partial o_i^{T,N}}{\partial u_i^{T,N}} = \delta_i^{T,N} \frac{\partial o_i^{T,N}}{\partial u_i^{T,N}}. \quad (12)$$

$$\delta_i^{t,n} = \frac{\partial L}{\partial o_i^{t,n}}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 1 是： $t=T$, $n=N$

最后一个时间步、输出层。

在这个位置，误差不需要从未来时间步或更高层反传回来，
因为：

- 已经是最后时间步 T ，没有 $t+1$
- 已经是输出层 N ，没有 $n+1$

所以这里的梯度可以直接由损失函数 L 得到。

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

$$L = \frac{1}{2S} \sum_{s=1}^S \left\| y_s - \frac{1}{T} \sum_{t=1}^T o_s^{t,N} \right\|^2$$

输出层不是只看某一个时刻的 spike，而是看 T 个时间步内输出 spike 的平均发放率。

对第 s 个样本，输出层第 i 个神经元：

$$L = \frac{1}{2S} \sum_{s=1}^S \sum_i \left(y_{s,i} - \frac{1}{T} \sum_{k=1}^T o_{s,i}^{k,N} \right)^2$$

只看与 $o_{s,i}^{T,N}$ 有关的那一项：

$$\frac{\partial L}{\partial o_{s,i}^{T,N}} = \frac{\partial}{\partial o_{s,i}^{T,N}} \frac{1}{2S} \left(y_{s,i} - \frac{1}{T} \sum_{k=1}^T o_{s,i}^{k,N} \right)^2$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

所以:

$$\frac{\partial L}{\partial o_{s,i}^{T,N}} = -\frac{1}{TS} \left(y_{s,i} - \frac{1}{T} \sum_{k=1}^T o_{s,i}^{k,N} \right)$$

如果省略样本索引s, 就是论文里的形式:

$$\frac{\partial L}{\partial o_i^{T,N}} = -\frac{1}{TS} \left(y_i - \frac{1}{T} \sum_{k=1}^T o_i^{k,N} \right)$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 2: $t = T$ at the layers $n < N$.

In this case, the derivative $\frac{\partial L}{\partial o_i^{T,n}}$ iteratively depends on the error propagation in the SD at time T as the typical BP algorithm. We have

$$\frac{\partial L}{\partial o_i^{T,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{T,n+1} \frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{T,n+1} \frac{\partial g}{\partial u_i^{T,n}} w_{ji}.$$

Similarly, the derivative $\frac{\partial L}{\partial u_i^{T,n}}$ yields

$$\frac{\partial L}{\partial u_i^{T,n}} = \frac{\partial L}{\partial u_i^{T+1,n}} \frac{\partial u_i^{T+1,n}}{\partial u_i^{T,n}} = \frac{\partial L}{\partial u_i^{T+1,n}} f(o_i^{T+1,n}). \quad (14)$$

是这样吗？

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 2 是: $t = T, n < N$

最后一个时间步, 但不是输出层。

因此:

- 没有未来时间步 $T+1$ 的时间反传;
- 会从下一层 $n+1$ 接收空间反传误差。

所以 Case 2 只考虑 **SD, Spatial Domain**, 空间域反传, 类似普通 DNN 的逐层反向传播。

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

论文中,
$$\frac{\partial L}{\partial o_i^{T,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{T,n+1} \frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}}$$

因为 $o_i^{T,n}$ 连接到下一层多个神经元:

$$o_i^{T,n} \rightarrow u_1^{T,n+1}, u_2^{T,n+1}, \dots, u_{l(n+1)}^{T,n+1}$$

而每个下一层神经元又产生:

$$o_j^{T,n+1} = g(u_j^{T,n+1})$$

第n层第i个神经元的输出 $o_i^{T,n}$, 会影响第n+1层所有神经元的输出 $o_j^{T,n+1}$, 所以它的误差要从下一层所有神经元累加回来。

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

所以总误差是所有路径贡献之和：

$$\frac{\partial L}{\partial o_i^{T,n}} = \sum_j \frac{\partial L}{\partial o_j^{T,n+1}} \frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}}$$

定义：

$$\delta_j^{T,n+1} = \frac{\partial L}{\partial o_j^{T,n+1}}$$

可得到：

$$\frac{\partial L}{\partial o_i^{T,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{T,n+1} \frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

展开 $\frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}}$:

下一层输出满足: $o_j^{T,n+1} = g(u_j^{T,n+1})$

所以:

$$\frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}} = \frac{\partial g}{\partial u_j^{T,n+1}} \frac{\partial u_j^{T,n+1}}{\partial o_i^{T,n}}$$

前向传播中:

$$x_j^{T,n+1} = \sum_{i=1}^{l(n)} w_{ji}^{n+1} o_i^{T,n}$$

所以: $\frac{\partial u_j^{T,n+1}}{\partial o_i^{T,n}} = w_{ji}^{n+1}$

$$\frac{\partial o_j^{T,n+1}}{\partial o_i^{T,n}} = \frac{\partial g}{\partial u_j^{T,n+1}} w_{ji}^{n+1}$$

$$u_j^{T,n+1} = u_j^{T-1,n+1} f(o_j^{T-1,n+1}) + x_j^{T,n+1} + b_j^{n+1}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 2 $t = T$, $n < N$ 时:

可得
$$\frac{\partial L}{\partial o_i^{T,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{T,n+1} \frac{\partial g}{\partial u_j^{T,n+1}} w_{ji}^{n+1}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

下面看Case 2 $t = T, n < N$ 中的 $\frac{\partial L}{\partial u_i^{T,n}}$:

既然 $t=T$, 没有未来时间项, 那么 $\frac{\partial L}{\partial u_i^{T,n}}$ 应该来自当前时刻 spike输出:

$$u_i^{T,n} \rightarrow o_i^{T,n} \rightarrow \text{下一层}$$

因此:

$$\frac{\partial L}{\partial u_i^{T,n}} = \frac{\partial L}{\partial o_i^{T,n}} \frac{\partial o_i^{T,n}}{\partial u_i^{T,n}}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 3: $t < T$ at the output layer $n = N$.

In this case, the derivative $\frac{\partial L}{\partial o_i^{t,N}}$ depends on the error propagation in the TD direction. With the help of the proposed iterative LIF model in Eq.(3)-(5) by unfolding the state space in the TD, we acquire the required derivative based on the chain rule in the TD as follows

$$\frac{\partial L}{\partial o_i^{t,N}} = \delta_i^{t+1,N} \frac{\partial o_i^{t+1,N}}{\partial o_i^{t,N}} + \frac{\partial L}{\partial o_i^{T,N}} \quad (15)$$

$$= \delta_i^{t+1,N} \frac{\partial g}{\partial u_i^{t+1,N}} u_i^{t,N} \frac{\partial f}{\partial o_j^{t,N}} + \frac{\partial L}{\partial o_i^{T,N}}, \quad (16)$$

$$\frac{\partial L}{\partial u_i^{t,N}} = \frac{\partial L}{\partial o_i^{t,N}} \frac{\partial o_i^{t,N}}{\partial u_i^{t,N}} = \delta_i^{t,N} \frac{\partial g}{\partial u_i^{t,N}}, \quad (17)$$

where $\frac{\partial L}{\partial o_i^{T,N}} = -\frac{1}{TS}(y_i - \frac{1}{T} \sum_{k=1}^T o_i^{k,N})$ as in Eq.(11).

是这样吗？

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 3 是: $t < T$, $n = N$

输出层, 但不是最后一个时间步。

因此它没有来自下一层的空间误差, 因为已经是输出层 N 。

但是它有来自未来时间步的时间误差, 因为 $t < T$ 。

所以 **Case 3** 主要讨论 **TD** 方向, 即时间方向的反向传播。

Case 3 的误差主要来自两部分:

- 当前输出 $o_i^{t,N}$ 对损失函数的直接影响;
- 当前状态对下一时刻 $u_i^{t+1,N}$ 的影响。

$$\frac{\partial L}{\partial o_i^{t,N}} = \delta_i^{t+1,N} \frac{\partial o_i^{t+1,N}}{\partial o_i^{t,N}} + \frac{\partial L}{\partial o_i^{T,N}}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

论文中,

$$\frac{\partial L}{\partial o_i^{t,N}} = \delta_i^{t+1,N} \frac{\partial o_i^{t+1,N}}{\partial o_i^{t,N}} + \frac{\partial L}{\partial o_i^{T,N}}$$

由链式法则: $\frac{\partial o_i^{t+1,N}}{\partial o_i^{t,N}} = \frac{\partial o_i^{t+1,N}}{\partial u_i^{t+1,N}} \frac{\partial u_i^{t+1,N}}{\partial o_i^{t,N}}$

$$\frac{\partial o_i^{t+1,N}}{\partial u_i^{t+1,N}} = g'(u_i^{t+1,N})$$

$$u_i^{t+1,N} = u_i^{t,N} f(o_i^{t,N}) + x_i^{t+1,N} + b_i^N$$

$$\frac{\partial u_i^{t+1,N}}{\partial o_i^{t,N}} = u_i^{t,N} f'(o_i^{t,N})$$

$$\frac{\partial L}{\partial o_i^{t,N}} = \delta_i^{t+1,N} g'(u_i^{t+1,N}) u_i^{t,N} f'(o_i^{t,N}) + \frac{\partial L}{\partial o_i^{T,N}}$$

Spatio-Temporal Backpropagation (STBP) 算法

2. 时空反向传播训练框架

Case 4: $t < T$ at the layers $n < N$.

In this case, the derivative $\frac{\partial L}{\partial o_i^{t,n}}$ depends on the error propagation in both SD and TD. On one side, each neuron accumulates the weighted error signals from the upper layer in the SD like Case 2; on the other side, each neuron also receives the propagated error from self-feedback dynamics in the TD by iteratively unfolding the state space based on the chain rule like Case 3. So we have

$$\frac{\partial L}{\partial o_i^{t,n}} = \sum_{j=1}^{l(n+1)} \delta_j^{t,n+1} \frac{\partial o_j^{t,n+1}}{\partial o_i^{t,n}} + \frac{\partial L}{\partial o_i^{t+1,n}} \frac{\partial o_i^{t+1,n}}{\partial o_i^{t,n}} \quad (18)$$

$$= \sum_{j=1}^{l(n+1)} \delta_j^{t,n+1} \frac{\partial g}{\partial u_i^{t,n}} w_{ji} + \delta_i^{t+1,n} \frac{\partial g}{\partial u_i^{t,n}} u_i^{t,n} \frac{\partial f}{\partial o_i^{t,n}}, \quad (19)$$

$$\frac{\partial L}{\partial u_i^{t,n}} = \frac{\partial L}{\partial o_i^{t,n}} \frac{\partial o_i^{t,n}}{\partial u_i^{t,n}} + \frac{\partial L}{\partial u_i^{t+1,n}} \frac{\partial u_i^{t+1,n}}{\partial u_i^{t,n}} \quad (20)$$

$$= \delta_i^{t,n} \frac{\partial g}{\partial u_i^{t,n}} + \frac{\partial L}{\partial u_i^{t+1,n}} f(o_i^{t+1,n}). \quad (21)$$

是这样吗？

写出STBP中Case1, Case2, Case3, Case4的正确形式。

下周二上课时交。

Spatio-Temporal Backpropagation (STBP) 算法

3. 代理梯度函数

关于对b和W的导数，有下面的式子：

$$\frac{\partial L}{\partial \mathbf{b}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}} \frac{\partial \mathbf{u}^{t,n}}{\mathbf{b}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}},$$
$$\frac{\partial L}{\partial \mathbf{W}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}} \frac{\partial \mathbf{u}^{t,n}}{\mathbf{W}^n} = \sum_{t=1}^T \frac{\partial L}{\partial \mathbf{u}^{t,n}} \mathbf{o}^{t,n-1T},$$

给定 STBP 中的 W 和 b，我们可以使用梯度下降优化算法有效地训练 SNN，从而获得高性能。

Spatio-Temporal Backpropagation (STBP) 算法

3. 代理梯度函数

脉冲不可微问题：神经元激活函数 $g(u)$ 是 $\delta(u)$ 的不可微狄拉克函数，该函数的值除了在零点处无穷大以外，其他地方的值都为零，这就导致了梯度消失或爆炸问题，从而使得误差误差传播。

➤ 为了解决该问题，STBP算法引入了四条曲线来用于近似表示脉冲激活函数的导数。

